

# End-to-End Test Report — Load Negotiation

## Playwright, real UI bundle, backend mocked at the network boundary

---

**Team:** Platform Engineering **Date:** 2026-07-03 **Framework:** Playwright 1.57 (Chromium) **Branch:** platform/SCRUM-248-esign-at-assign **Result:** 14 / 14 scenarios pass, 0 flaky, 17.2s **Scope:** the negotiation flow across both party UIs — the Owner-Operator Load Detail page ( /owner-operator/loads/:loadId , party = HAULER) and the Shipper Load Detail page ( /shipper/loads/:loadId , party = SHIPPER), driving the real NegotiationPanel .

---

## 1. Approach — and why it is hermetic

---

The suite drives the **real compiled React application** (Vite dev server on :3001 ) and the **real NegotiationPanel component** end to end: real routing, the real auth-gated RequireRole , the real long-poll live-update loop, the real buttons and state rendering. Only the **backend is mocked, at the network boundary** ( page.route( '\*\*/api/\*\*' , ... ) ) by a small stateful re-implementation of the negotiation state machine that mirrors the server's viewFor() and advances on the panel's POSTs.

### Why the backend is mocked rather than live:

- **It must not touch production.** Running against prod would mutate the demo load you will click through — every accept would assign it, every reject would rebroadcast it. The mock keeps prod pristine (the standing "don't change anything" constraint).
  - **The local Docker daemon is down**, so a full local DynamoDB + backend stack could not be brought up unattended tonight.
  - A network-boundary mock is a legitimate, deterministic E2E strategy: the UI, its API calls, and its rendered reactions are all real; only the server's responses are scripted. All external requests (Google Maps, fonts, analytics) are blocked so the run is fully offline and reproducible.
- 

## 2. What "100% coverage" means here

---

E2E does not measure line coverage — it measures **behaviour coverage**. The target is **100% of the UI-reachable transitions of the negotiation state machine**: every edge a real user can trigger from a button, plus the live-update channel and the e-sign gate. The matrix in §3 shows

every such edge with the scenario that exercises it. (Statement/branch/basis-path coverage of the underlying code is measured separately in the **Path Coverage report**; the two are complementary.)

The one state not driven by a button — **EXPIRED** — is reachable in the UI only through the long-poll delivering an expiry (there is no "expire" control); scenario **H9** exercises exactly that path, so the observable state machine is fully covered.

### 3. Transition coverage matrix

| #  | State transition                                                  | UI trigger                             | Party   | Scenario |
|----|-------------------------------------------------------------------|----------------------------------------|---------|----------|
| 1  | $\emptyset \rightarrow$ <b>ENGAGED</b>                            | Engage to negotiate                    | HAULER  | H1       |
| 2  | ENGAGED $\rightarrow$ <b>ACCEPTED</b> (posted rate)               | Accept load                            | HAULER  | H2       |
| 3  | ENGAGED $\rightarrow$ <b>PENDING_SHIPPER</b>                      | Bid                                    | HAULER  | H3       |
| 4  | ENGAGED $\rightarrow$ <b>REJECTED</b>                             | Reject                                 | HAULER  | H7       |
| 5  | PENDING_SHIPPER $\rightarrow$ <b>ACCEPTED</b> (bid)               | Accept bid                             | SHIPPER | S2       |
| 6  | PENDING_SHIPPER $\rightarrow$ <b>PENDING_HAULER</b>               | Counter                                | SHIPPER | S3       |
| 7  | PENDING_SHIPPER $\rightarrow$ <b>REJECTED</b>                     | Reject bid                             | SHIPPER | S4       |
| 8  | PENDING_HAULER $\rightarrow$ <b>ACCEPTED</b> (counter)            | Accept counter                         | HAULER  | H5       |
| 9  | PENDING_HAULER $\rightarrow$ <b>PENDING_SHIPPER</b>               | Counter offer                          | HAULER  | H6       |
| 10 | active $\rightarrow$ <b>EXPIRED</b> (window elapsed)              | long-poll delivery                     | HAULER  | H9       |
| 11 | <b>live update</b> (counterparty move via long-poll)              | events channel                         | both    | H4, S5   |
| 12 | <b>e-sign gate</b> : assign 412-blocked, then allowed once signed | Accept (blocked $\rightarrow$ allowed) | HAULER  | H8       |
| 13 | <b>read model</b> renders correct status + actions per party      | initial render                         | both    | H1, S1   |

Every UI-reachable transition is covered. **13 of 13 matrix rows  $\rightarrow$  covered (100%).**

### 4. Results

All 14 scenarios passed on the same run (no retries consumed):

| Scenario           | What it proves                                                  | Time |
|--------------------|-----------------------------------------------------------------|------|
| H1 engage          | hauler holds the load; sees Accept load / Bid / Reject          | 0.4s |
| H2 accept-load     | takes the posted rate → "Assigned at \$2.50/mi"                 | 1.3s |
| H3 bid             | cents-per-mile bid → pending on the shipper, offer on the table | 0.5s |
| H4 live update     | the shipper's counter arrives via the long-poll (~1s)           | 0.8s |
| H5 accept-counter  | accepts the shipper's counter → assigned at \$2.60/mi           | 6.2s |
| H6 hauler counter  | counters back → pending on the shipper again                    | 1.4s |
| H7 reject          | walks away → load rebroadcasts                                  | 0.5s |
| H8 e-sign gate     | assign refused (412) with no signature; assigns once signed     | 2.1s |
| H9 expiry          | window elapses → terminal state arrives via long-poll           | ~1s  |
| S1 shipper view    | sees the hauler's bid + Accept bid / Counter / Reject           | 1.1s |
| S2 accept bid      | assigns at the hauler's rate → "Assigned at \$2.75/mi"          | 2.0s |
| S3 shipper counter | counters → pending on the hauler                                | 1.2s |
| S4 shipper reject  | rejects the bid → load rebroadcasts                             | 1.2s |
| S5 live update     | a hauler's bid appears while the shipper watches                | 1.1s |

Machine-readable results: `docs/overnight-2026-07-03/e2e/results.json` . Full Playwright HTML report (traces on any failure): `docs/overnight-2026-07-03/e2e/playwright-report/index.html` .

## 5. Findings surfaced by the E2E (with COA)

Building a real-browser E2E flushed out three issues that unit tests could not. None blocks the negotiation feature; all are worth a fast-follow.

### F1 — Onboarding tour overlay intercepts clicks on the load-detail panels (*UX / medium*)

The Shepherd persona tour auto-starts ~700ms after a dashboard-family route mounts. On the Owner-Operator and Shipper **load-detail** pages its modal overlay renders **on top of the negotiation panel**, intermittently intercepting clicks (it blocked the accept button mid-test until suppressed). A user who deep-links to a load while the tour is unseen can be blocked from acting on it. **COA:** do not auto-start the tour on deep-linked `.../loads/:id` routes (only on the dashboard root), or ensure the tour's overlay never covers primary action panels and dismisses on

outside interaction. (The E2E suppresses the tour via its own `localStorage` completion flag — the app's supported gate.)

## F2 — Hard page reload on assignment (UX / low)

Both detail pages wire `onAssigned={() => window.location.reload()}`. On accept, the whole page reloads — scroll position is lost, every panel refetches, and there is a visible flash. **COA:** refetch just the load + negotiation state (a local `refresh()`), not a full document reload.

## F3 — Panels white-screen on a malformed list response (resilience / low)

`AccessorialsPanel` reads `res.charges` and `AttestationChain` reads `res.chain` and then render `.length / .map` on the result. When the response lacks that array (an unexpected shape or a partial outage), the value is `undefined` and the **whole page** crashes to the "Something went wrong" error boundary — the negotiation panel included, because they share the page. (Surfaced when the mock returned a bare object; the live backend returns the correct shape today.) **COA:** default at the read site — `const charges = res?.charges ?? [] / res?.chain ?? []` — so an unexpected response degrades to an empty list instead of taking down the page.

---

## 6. How to run

```
cd frontend-v2
npm run test:e2e           # boots vite on :3001, runs all specs headless
npm run test:e2e:report    # opens the HTML report
```

Chromium is installed via `npx playwright install chromium`. The suite is hermetic (no network, no prod).

---

## Appendix — harness design

- `frontend-v2/playwright.config.ts` — vite `webServer` on `:3001`, retries 2 (absorbs long-poll timing jitter), reports to the deliverables folder.
- `frontend-v2/e2e/support/negotiationMock.ts` — the stateful network-boundary mock: it implements `viewFor()` per party, advances the state machine on each POST, holds the events long-poll (~500ms) to emulate the server and avoid a tight loop, exposes a controller to simulate the counterparty (`shipperCounter`, `haulerBid`, `expire`) and the e-sign gate

( `setEsignBlocked` ), suppresses the persona tour, and returns benign empty shapes for incidental endpoints so the page always renders.

- `frontend-v2/e2e/haulerFlows.spec.ts` (H1–H9) and `frontend-v2/e2e/shipperFlows.spec.ts` (S1–S5).

**Not deployed. Branch artifact for review.**